



Micro Frontends Integration

In this lesson, we will discuss how micro frontends are integrated with each other to make the complete UI of a website.

We'll cover the following



- Client-side integration of micro frontends
- Server-side integration
- Technology and frameworks facilitating server-side integration

Once we have respective micro frontends ready for our online game store, we need to integrate them together to have a functional website. There are two ways we can do this -

1. By integrating micro frontends on the client
2. By integrating micro frontends on the server

This concept is similar to the *client-side* and *server-side rendering*, which we discussed earlier in the course.

In this micro frontend scenario, we need to write additional logic to enable the integration of the UI components.

Let's discuss the *client-side integration* process first.



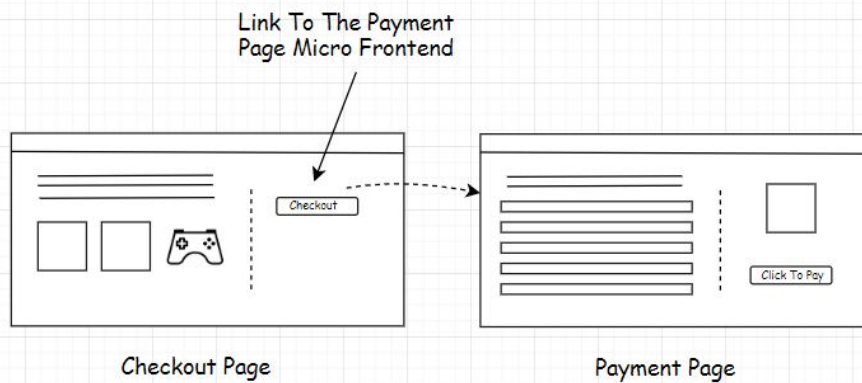
Client-side integration of micro frontends



A very basic naïve way of integrating components on the client is rendering micro frontends with unique links. We just place the links on the website to



enable the user to navigate to a certain micro frontend, like so:



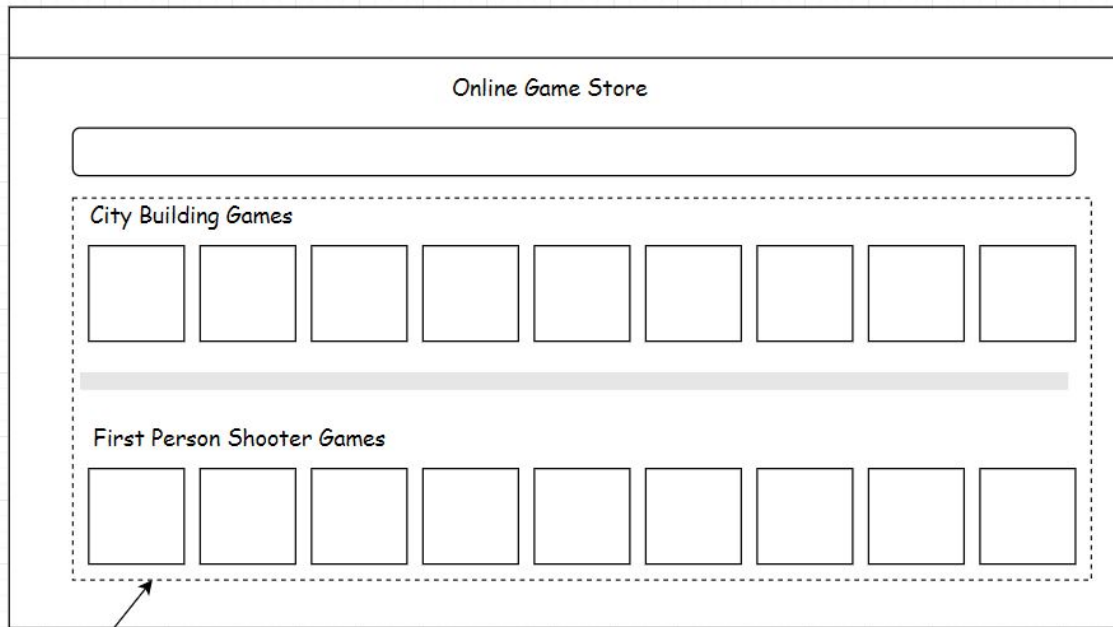
8bitmen.com

Consider this scenario - our order checkout microservice is hosted on AWS having the URL - <https://www.aws.amazon.com/onlinegamestore/checkout> and our payment service is hosted on Google Cloud with the URL <https://www.cloud.google.com/onlinegamestore/payment>

When we integrate these micro frontends via basic links and the user navigates from the checkout page to the payment page clicking on the micro frontend link, the address in the user's browser changes from the AWS URL to the Google Cloud URL. This will be visible to the end-user.

Also, following this basic approach, the micro frontends that need to be integrated within a specific page can be done using *Iframes*.





Game Category Fragment Integrated Into The Home Page Of The Website Using An Iframe

8bitmen.com

Well, you would have realized that this approach is not ideal. This is more like the 90s way of building websites, connecting web pages via *direct links* and *Iframes*.

Using *IFrames* has several downsides: they can't be bookmarked, they aren't good from an SEO standpoint, they have stability and performance issues and so on. Besides, IFrames are an obsolete thing in the modern web dev landscape.

A recommended way to integrate components on the client-side is by leveraging a technology called the Web Components and frameworks such as [Single SPA](#).

Single SPA is a *JavaScript* framework for frontend microservices that enable developers to build their frontend while leveraging different JavaScript frameworks.

If you wish to know more on web components, this [Google Chrome devs video](#) is a good resource.

Let's move on to understand the server-side integration process of micro frontends.

> Server-side integration

When the UI components are integrated on the server, on the user request, the complete pre-built page of the website is delivered to the client from the server as opposed to sending individual micro frontends to the client and having them clubbed there.

This cuts down the website's loading time on the client significantly since the browser does not have to do any sort of heavy lifting.

Just like the client-side integration process, we have to write additional logic on the server to integrate the micro frontends that are requested by the user.

There are several technologies and frameworks available that help us to achieve this.

Technology and frameworks facilitating server-side integration

Some of the popular frameworks that facilitate server-side integration of micro frontends are Project Mosaic, Open Components and Podium. Server Side Includes SSI is a server-side scripting language used for clubbing the content of multiple web pages on the webserver.

Zalando is a famous fashion e-commerce company in Europe that uses Project Mosaic to manage its micro frontends. Here is a talk on the micro frontends from their engineering team - [The Zalando recipe for scalable frontends](#).



Well, this pretty much sums up our micro frontends chapter. Let's move on to databases in the next one.

